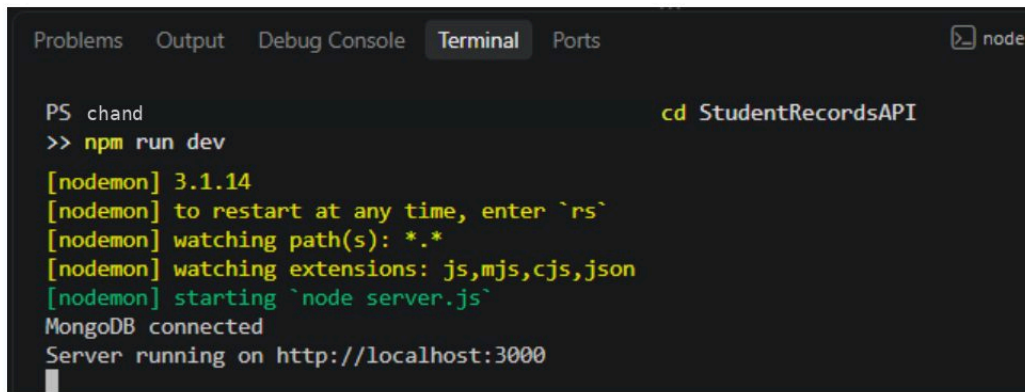


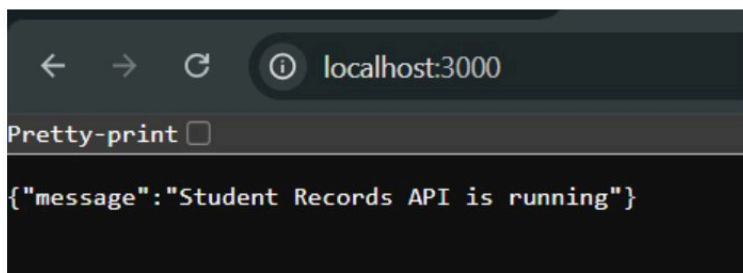
DBMS Practical – 7

SERVER running:



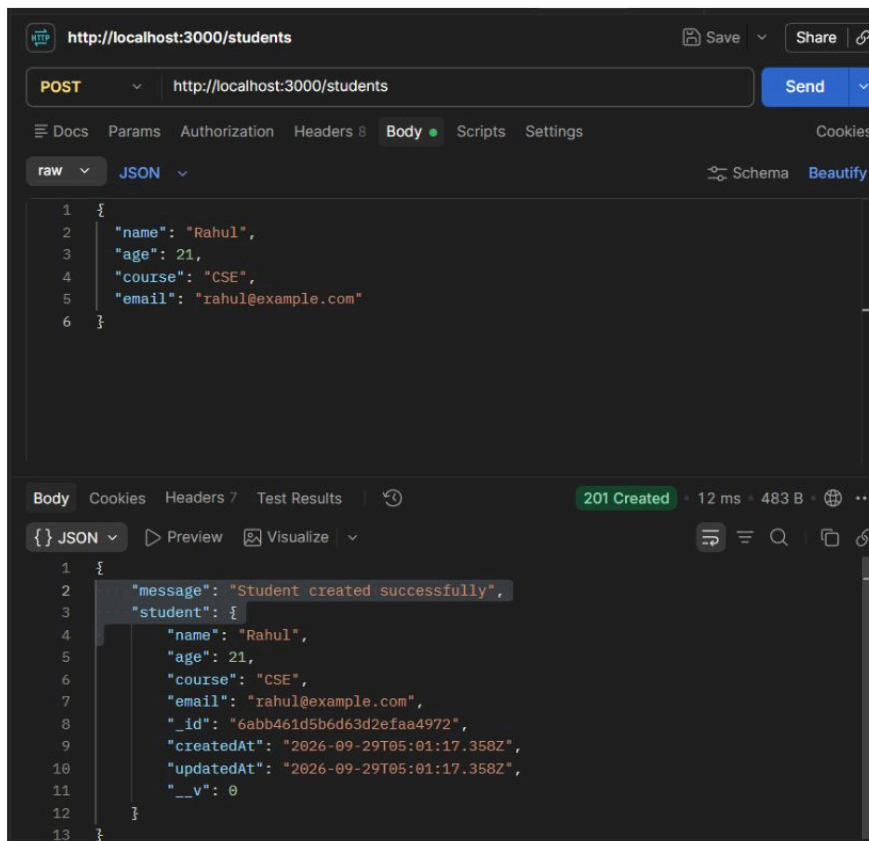
```
Problems Output Debug Console Terminal Ports node
PS chand
>> npm run dev
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
MongoDB connected
Server running on http://localhost:3000
```

API Running:



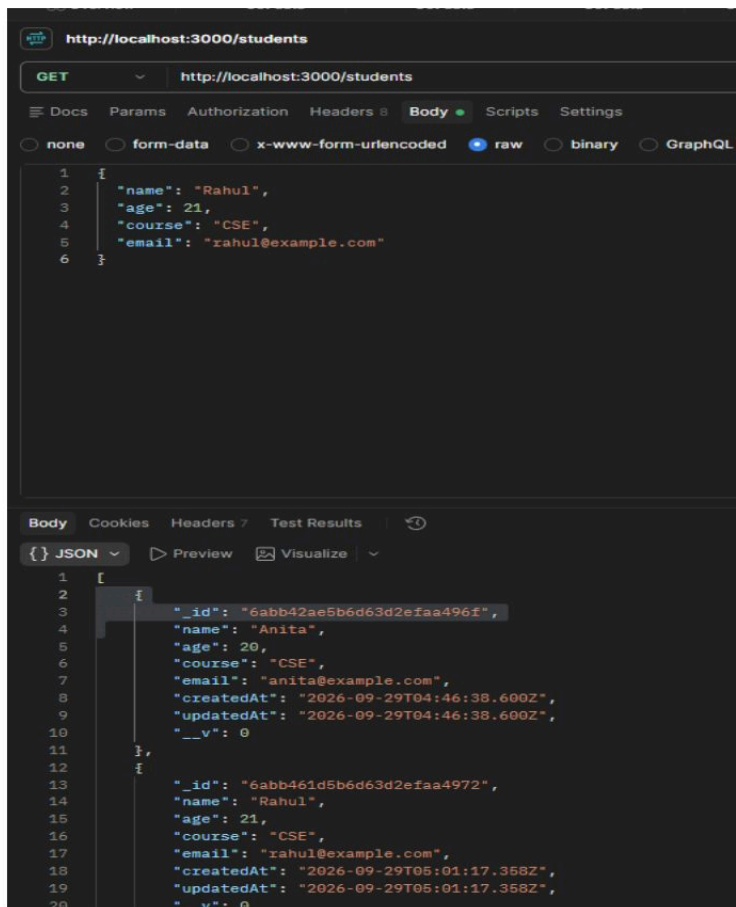
```
localhost:3000
Pretty-print
{"message": "Student Records API is running"}
```

POST:



```
http://localhost:3000/students
POST http://localhost:3000/students
Body
raw JSON
1 {
2   "name": "Rahul",
3   "age": 21,
4   "course": "CSE",
5   "email": "rahul@example.com"
6 }
201 Created - 12 ms - 483 B
Body
1 {
2   "message": "Student created successfully",
3   "student": {
4     "name": "Rahul",
5     "age": 21,
6     "course": "CSE",
7     "email": "rahul@example.com",
8     "_id": "6abb461d5b6d63d2efaa4972",
9     "createdAt": "2026-09-29T05:01:17.358Z",
10    "updatedAt": "2026-09-29T05:01:17.358Z",
11    "__v": 0
12  }
13 }
```

GET:



http://localhost:3000/students

GET http://localhost:3000/students

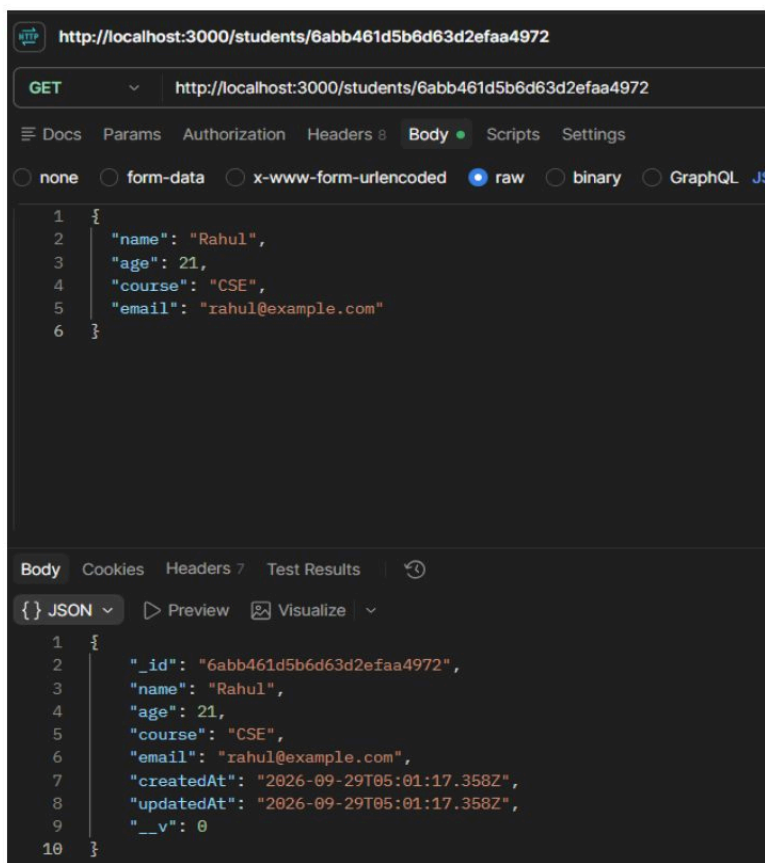
Body

```
1 {
2   "name": "Rahul",
3   "age": 21,
4   "course": "CSE",
5   "email": "rahul@example.com"
6 }
```

Body

JSON

```
1 [
2   {
3     "_id": "6abb42ae5b6d63d2efaa496f",
4     "name": "Anita",
5     "age": 20,
6     "course": "CSE",
7     "email": "anita@example.com",
8     "createdAt": "2026-09-29T04:46:38.600Z",
9     "updatedAt": "2026-09-29T04:46:38.600Z",
10    "__v": 0
11  },
12  {
13    "_id": "6abb461d5b6d63d2efaa4972",
14    "name": "Rahul",
15    "age": 21,
16    "course": "CSE",
17    "email": "rahul@example.com",
18    "createdAt": "2026-09-29T05:01:17.358Z",
19    "updatedAt": "2026-09-29T05:01:17.358Z",
20    "v": 0
21  }
22 ]
```



http://localhost:3000/students/6abb461d5b6d63d2efaa4972

GET http://localhost:3000/students/6abb461d5b6d63d2efaa4972

Body

```
1 {
2   "name": "Rahul",
3   "age": 21,
4   "course": "CSE",
5   "email": "rahul@example.com"
6 }
```

Body

JSON

```
1 {
2   "_id": "6abb461d5b6d63d2efaa4972",
3   "name": "Rahul",
4   "age": 21,
5   "course": "CSE",
6   "email": "rahul@example.com",
7   "createdAt": "2026-09-29T05:01:17.358Z",
8   "updatedAt": "2026-09-29T05:01:17.358Z",
9   "__v": 0
10 }
```

PATCH:

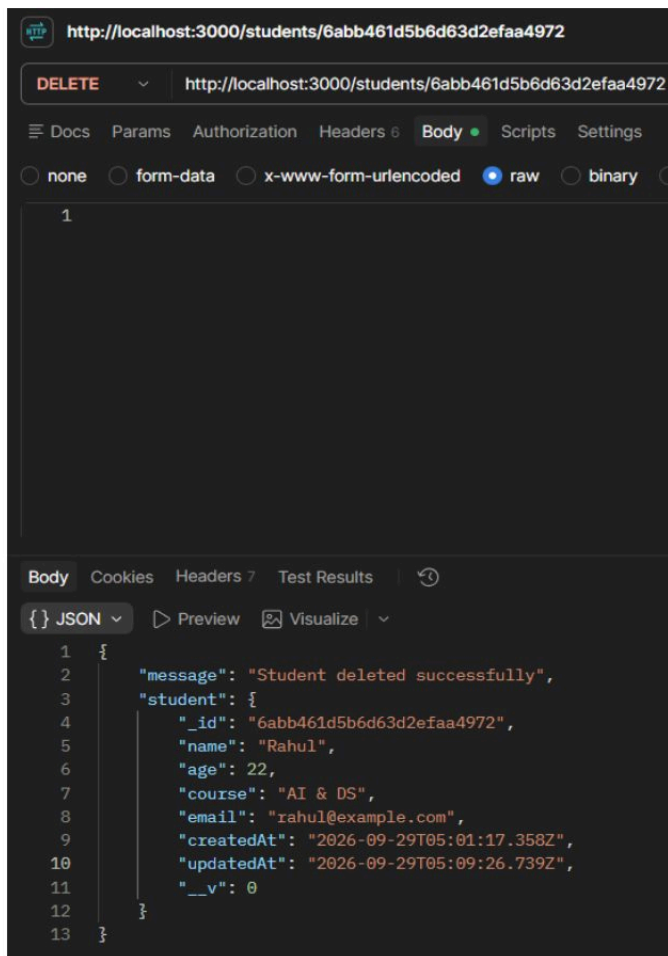
The screenshot displays a REST client interface with the following components:

- URL Bar:** Shows the URL `http://localhost:3000/students/6abb461d5b6d63d2efaa4972`.
- Method:** A dropdown menu is set to `PATCH`.
- Body Tab:** The `Body` tab is selected, showing the request body in `raw` format. The body is a JSON object:

```
1 {  
2   "age": 22,  
3   "course": "AI & DS"  
4 }
```
- Response Section:** Below the request, the `Body` tab is selected for the response. The response is in `JSON` format and shows a successful update:

```
1 {  
2   "message": "Student updated successfully",  
3   "student": {  
4     "_id": "6abb461d5b6d63d2efaa4972",  
5     "name": "Rahul",  
6     "age": 22,  
7     "course": "AI & DS",  
8     "email": "rahul@example.com",  
9     "createdAt": "2026-09-29T05:01:17.358Z",  
10    "updatedAt": "2026-09-29T05:07:57.556Z",  
11    "__v": 0  
12  }  
13 }
```

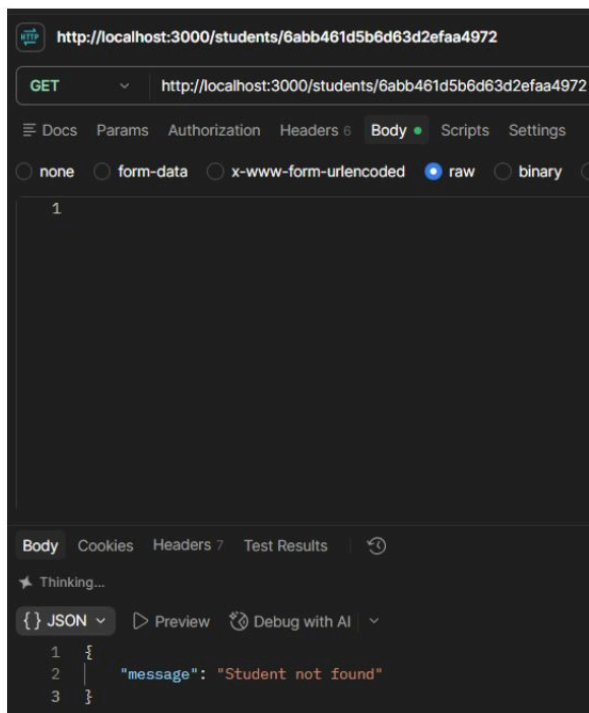
DELETE:



A screenshot of the Postman application showing a DELETE request to the URL `http://localhost:3000/students/6abb461d5b6d63d2efaa4972`. The request method is set to **DELETE**. The **Body** tab is selected, and the content type is set to **raw**. The body contains the number `1`. The **Test Results** tab is also visible, showing a successful response with a status of 200. The response body is displayed in the **Body** tab, showing a JSON object with a success message and student details.

```
{
  "message": "Student deleted successfully",
  "student": {
    "_id": "6abb461d5b6d63d2efaa4972",
    "name": "Rahul",
    "age": 22,
    "course": "AI & DS",
    "email": "rahul@example.com",
    "createdAt": "2026-09-29T05:01:17.358Z",
    "updatedAt": "2026-09-29T05:09:26.739Z",
    "__v": 0
  }
}
```

GET (VERIFY):



A screenshot of the Postman application showing a GET request to the URL `http://localhost:3000/students/6abb461d5b6d63d2efaa4972`. The request method is set to **GET**. The **Body** tab is selected, and the content type is set to **raw**. The body contains the number `1`. The **Test Results** tab is also visible, showing a successful response with a status of 200. The response body is displayed in the **Body** tab, showing a JSON object with a success message and student details.

```
{
  "message": "Student not found"
}
```

Duplicate Email Validation:

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:3000/students`
- Method:** `POST`
- Body (raw):**

```
1 {
2   "name": "Another Anita",
3   "age": 22,
4   "course": "ECE",
5   "email": "anita@example.com"
6 }
```
- Status:** `400 Bad Request` (6 ms, 416 B)
- Response Body (JSON):**

```
1 {
2   "message": "Failed to create student",
3   "error": "E11000 duplicate key error collection: student_records.students index: email_1 dup
4             key: { email: \"anita@example.com\" }"
5 }
```

MongoDB Compass:

The screenshot shows the MongoDB Compass interface with the following details:

- Database:** `DBMS > student_records > students`
- Tab:** `Documents` (1 document)
- Query:** `Type a query: { field: 'value' } or Generate query`
- Buttons:** `ADD DATA`, `BULK`, `EXPORT DATA`, `EXPORT CODE`
- Document Details:**

```
_id: ObjectId('6abb42ae5b6d63d2efaa496f')
name: "Anita"
age: 20
course: "CSE"
email: "anita@example.com"
createdAt: ISODate('2026-09-29T04:46:38.600+00:00') September 29, 2026 at 4:46:38 AM UTC
updatedAt: ISODate('2026-09-29T04:46:38.600+00:00') September 29, 2026 at 4:46:38 AM UTC
__v: 0
```